

Chapter 8 Clustering

Austin R.J. Downey

Clustering

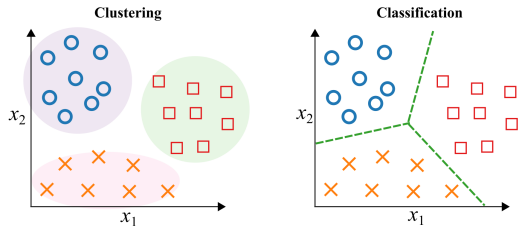
Clustering groups observations that “look similar” without using labels.

Key idea:

- ▶ No target vector $\mathbf{y}$
- ▶ The algorithm discovers structure
- ▶ Similar points are grouped together

Useful when you:

- ▶ have no ground-truth labels
- ▶ suspect hidden sub-populations
- ▶ want exploratory structure



Classification uses labels; clustering does not.

Review: Wine Dataset

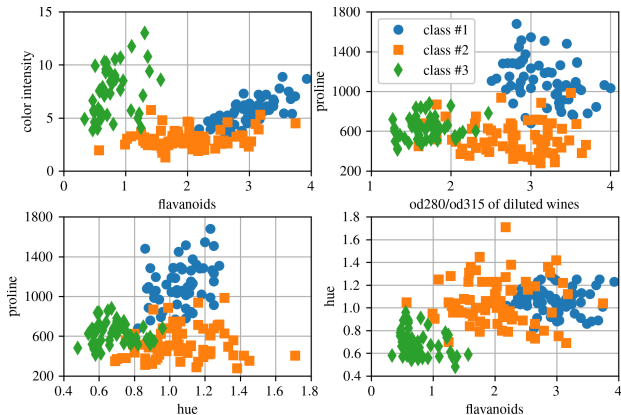
The wine dataset contains:

- ▶ 178 wine samples
- ▶ 3 known cultivars
- ▶ 13 continuous measurements

Examples of features:

- ▶ alcohol
- ▶ flavonoids
- ▶ magnesium
- ▶ color intensity

The labels are useful for checking results, but clustering does not use them.



Feature-feature plots for the wine data.

What a Clustering Algorithm Sees

The input is the feature matrix:

$$X = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^T$$

Each observation is a feature vector:

$$\mathbf{x}_i \in \mathbb{R}^d$$

There is no target vector $\mathbf{y}$.

Most clustering methods depend on a distance or similarity measure.

A common choice is Euclidean distance:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2$$

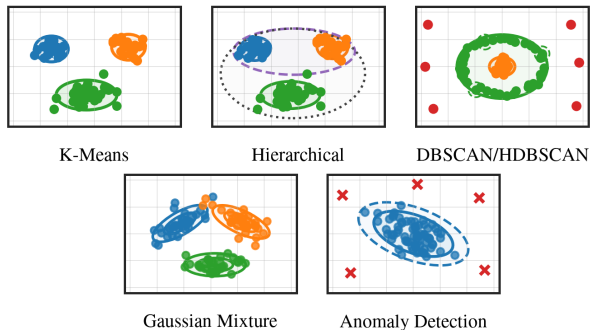
Feature scaling matters because large-range features can dominate this distance.

`StandardScaler` is often a good default.

Different Methods Define “Cluster” Differently

The main difference is the modeling assumption.

- ▶ K-means: center-based clusters
- ▶ Hierarchical: nested groups
- ▶ DBSCAN/HDBSCAN: dense regions
- ▶ Gaussian mixtures: probability distributions
- ▶ Anomaly detection: low-likelihood points



Common clustering ideas.

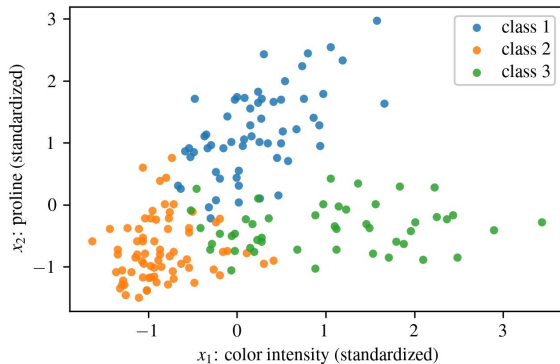
No single method is best for every dataset.

K-Means

K-means partitions data into a chosen number of clusters, k .

Each cluster is represented by a centroid.

The algorithm assigns each observation to the nearest centroid.



Wine data using two standardized features.

Objective Function

Given observations:

$$X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\} \subset \mathbb{R}^d$$

K-means seeks:

- ▶ centroids $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_k$
- ▶ assignments $c_i \in \{1, 2, \dots, k\}$

The objective is the within-cluster sum of squares:

$$J = \sum_{i=1}^n \|\mathbf{x}_i - \boldsymbol{\mu}_{c_i}\|^2$$

This is also called:

inertia

Small J means points are close to their assigned centroids.

The cluster label c_i is just an arbitrary index.

Assignment and Update Steps

Lloyd's algorithm alternates between two steps.

1. Assignment step

Assign each point to its closest centroid:

$$c_i = \arg \min_{j \in \{1, \dots, k\}} \|\mathbf{x}_i - \boldsymbol{\mu}_j\|^2$$

2. Update step

Move each centroid to the mean of the points assigned to it:

$$\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{i: c_i = j} \mathbf{x}_i$$

Repeat until assignments stop changing or J changes very little.

Decision Regions

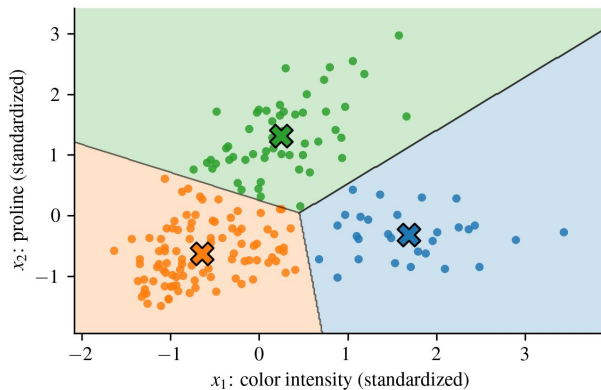
The assignment step divides feature space into regions.

Every point in a region is closest to the same centroid.

These regions are also called:

Voronoi cells

For K-means, the boundaries are determined by distance to centroids.



K-means decision regions for $k = 3$.

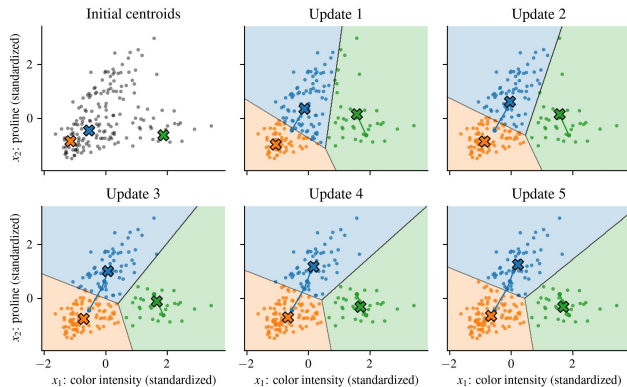
Centroid Movement

K-means starts from initial centroids.

Then it repeatedly:

- ▶ assigns points
- ▶ updates centroids
- ▶ lowers or keeps the same objective value

After several iterations, the centroids settle into stable positions.



Centroid movement during Lloyd iterations.

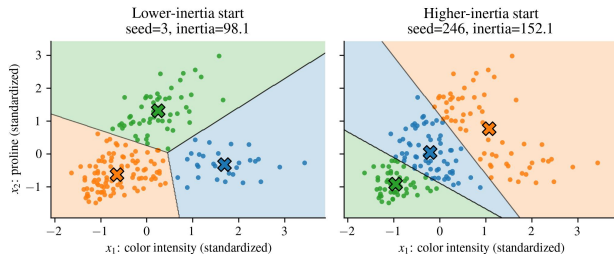
Random Starts and Local Minima

The K-means objective is not convex.

Different initial centroids can lead to different final clusters.

Practical fixes:

- ▶ run K-means multiple times
- ▶ keep the lowest-inertia solution
- ▶ use `k-means++` initialization



Different random starts can converge to different solutions.

Choosing the Number of Clusters

The number of clusters, k , must be chosen before fitting K-means.

If k is too small:

- ▶ distinct groups may be merged

If k is too large:

- ▶ one group may be split artificially

Elbow rule:

$$\Delta J(k) = J(k-1) - J(k)$$

Silhouette value for point i :

$$s_i = \frac{b_i - a_i}{\max(a_i, b_i)}$$

Interpretation:

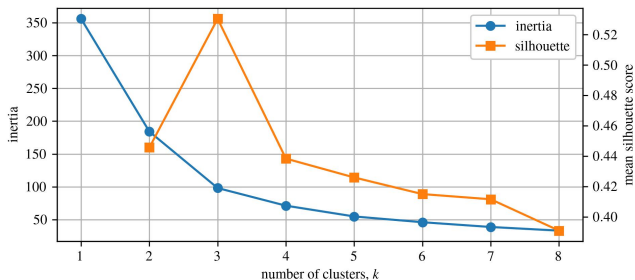
- ▶ near 1: well inside its cluster
- ▶ near 0: near a boundary
- ▶ negative: possibly assigned poorly

Cluster Diagnostics

Adding more clusters usually decreases inertia.

So we look for where the improvement begins to level off.

For the two-feature wine example, the diagnostics support $k = 3$.



Inertia and mean silhouette score for different values of k .

Practical Notes

K-means is usually efficient.

$$O(nkd)$$

where:

- ▶ n : observations
- ▶ k : clusters
- ▶ d : features

Practical defaults:

- ▶ scale continuous features
- ▶ use multiple random starts
- ▶ use `k-means++`
- ▶ interpret clusters as distance-based groups

K-means is not a classifier; it simply groups nearby points.

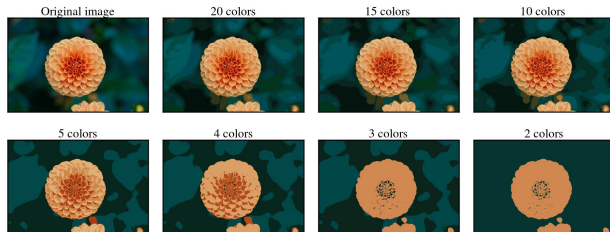
K-Means for Image Segmentation

K-means can cluster pixels.

For a color image, each pixel can be represented by:

$$\mathbf{x}_i = [R_i \quad G_i \quad B_i]^T$$

Replacing each pixel by its centroid color gives a segmented or compressed image.



Grouping pixels by similarity in color space.

Limitations of K-Means

K-means works best when clusters are:

- ▶ compact
- ▶ approximately spherical
- ▶ similar in size
- ▶ reasonably separated

K-means can perform poorly when clusters are:

- ▶ curved or nested
- ▶ elongated
- ▶ strongly overlapping
- ▶ very different in density
- ▶ affected by outliers

These are consequences of the centroid-and-distance assumption.

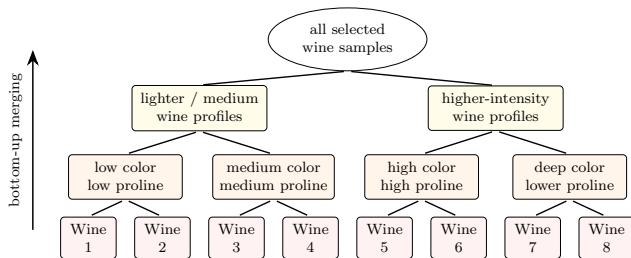
Hierarchical Agglomerative Clustering

Hierarchical clustering builds a nested sequence of clusters.

Agglomerative clustering is bottom-up:

- ▶ start with individual observations
- ▶ merge nearby groups
- ▶ continue until one cluster remains

The result is a hierarchy, not just one fixed k .



Representative bottom-up wine dendrogram.

Agglomerative Algorithm

The basic algorithm is simple to state:

1. Start with n clusters, one for each observation.
2. Compute distances between every pair of clusters.
3. Merge the two closest clusters.
4. Update distances to the new cluster.
5. Repeat until all observations have been merged into one cluster.

The key modeling decision is how to define the distance between two clusters.

distance between points $\implies$ distance between clusters

Linkage Rules

For clusters A and B :

Single linkage

$$D_{\text{single}}(A, B) = \min_{\mathbf{x}_i \in A, \mathbf{x}_j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$$

Complete linkage

$$D_{\text{complete}}(A, B) = \max_{\mathbf{x}_i \in A, \mathbf{x}_j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$$

Average linkage

$$D_{\text{average}}(A, B) = \frac{1}{|A||B|} \sum_{\mathbf{x}_i \in A} \sum_{\mathbf{x}_j \in B} d(\mathbf{x}_i, \mathbf{x}_j)$$

Interpretation:

- ▶ single: closest pair
- ▶ complete: farthest pair
- ▶ average: all pairwise distances

Ward Linkage

Ward linkage merges the pair of clusters that causes the smallest increase in within-cluster variation.

For cluster C :

$$W(C) = \sum_{\mathbf{x}_i \in C} \|\mathbf{x}_i - \boldsymbol{\mu}_C\|^2$$

When clusters A and B are merged:

$$\Delta W = W(A \cup B) - W(A) - W(B)$$

This is conceptually close to K-means because both methods favor compact clusters.

Dendrograms

A dendrogram is a tree diagram showing the order in which clusters are merged.

How to read it:

- ▶ bottom: individual observations
- ▶ upward movement: clusters merge
- ▶ merge height: dissimilarity between clusters
- ▶ horizontal cut: produces a chosen clustering solution

Why it is useful:

- ▶ inspect structure before choosing k
- ▶ large vertical gaps suggest natural cuts
- ▶ good for exploratory analysis and visualization

Limitation: pairwise distances can make it expensive for large datasets.

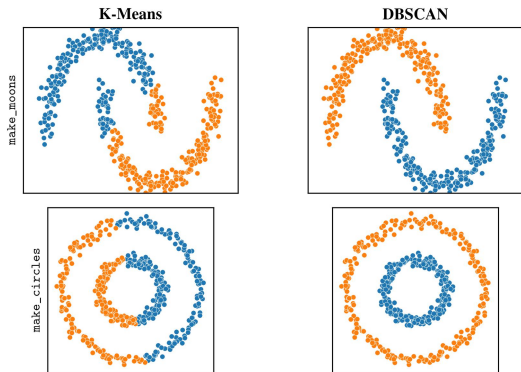
Density-Based Methods: DBSCAN and HDBSCAN

Density-based clustering defines clusters as dense regions separated by sparse regions.

This is different from K-means:

- ▶ no centroid is needed
- ▶ clusters can be curved
- ▶ outliers can be labeled as noise

The goal is to follow connected dense regions.



K-means versus DBSCAN on non-spherical data.

DBSCAN Parameters

DBSCAN has two main parameters:

- ▶ ϵ : neighborhood radius
- ▶ minPts: minimum number of nearby points

In scikit-learn:

`minPts` $\leftrightarrow$ `min_samples`

The ϵ -neighborhood is:

$$N_{\epsilon}(\mathbf{x}_i) = \{\mathbf{x}_j \in X : d(\mathbf{x}_i, \mathbf{x}_j) \leq \epsilon\}$$

A point is a core point if:

$$|N_{\epsilon}(\mathbf{x}_i)| \geq \text{minPts}$$

Core, Border, and Noise Points

DBSCAN separates observations into three roles.

Core point

- ▶ dense neighborhood
- ▶ at least minPts points nearby
- ▶ expands the cluster

Border point

- ▶ not dense enough by itself
- ▶ close to a core point
- ▶ attached to a cluster

Noise point

- ▶ not a core point
- ▶ not close enough to a core point
- ▶ remains unassigned

Clusters form by connecting chains of nearby core points.

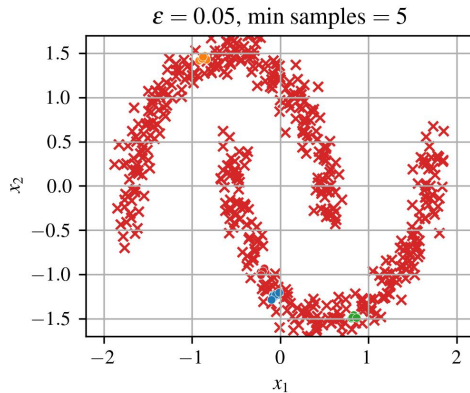
When ϵ Is Too Small

The choice of ϵ sets the density scale.

If ϵ is too small:

- ▶ neighborhoods contain too few points
- ▶ few points become core points
- ▶ many observations become noise

This does not mean the data have no structure; the density scale is wrong.



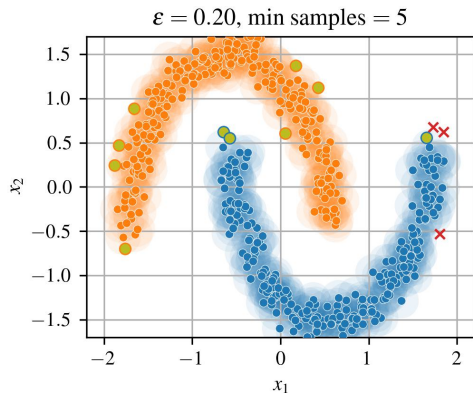
DBSCAN with ϵ too small.

A More Useful ϵ

With a more useful ϵ :

- ▶ neighborhoods overlap
- ▶ core points connect along the moons
- ▶ two curved clusters are recovered
- ▶ a few isolated points remain noise

DBSCAN works well when clusters are connected dense regions.



Overlapping neighborhoods connect core points.

Choosing ϵ and Moving to HDBSCAN

A common diagnostic is a nearest-neighbor distance plot.

Procedure:

- ▶ compute distance to the k -th nearest neighbor
- ▶ choose k near minPts
- ▶ sort distances from small to large
- ▶ look for a sharp bend

HDBSCAN reduces the need for one fixed ϵ .

It builds a hierarchy over many density levels and keeps clusters that are stable.

This helps when different clusters have different densities.

When to Use Density-Based Clustering

Density-based clustering is a good choice when:

- ▶ clusters may have irregular shapes
- ▶ the data contain outliers
- ▶ dense operating states are separated by sparse transitions
- ▶ the number of clusters is not obvious

DBSCAN is easiest to explain because ϵ and minPts have direct meanings.

HDBSCAN is often more flexible when clusters have different densities.

In both cases, feature scaling and the distance metric matter.

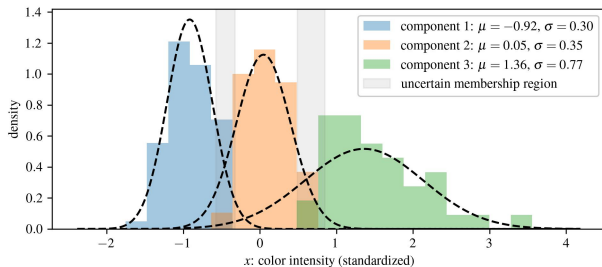
Gaussian Mixture Models

K-means assigns each observation to exactly one cluster.

Gaussian mixture models provide a probabilistic alternative.

The data are modeled as a weighted combination of Gaussian components.

Overlap between components becomes membership uncertainty.



One-dimensional GMM idea using wine color intensity.

Gaussian Mixture Density

A Gaussian mixture model with k components has density:

$$p(\mathbf{x}) = \sum_{j=1}^k \pi_j \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$$

The parameters are:

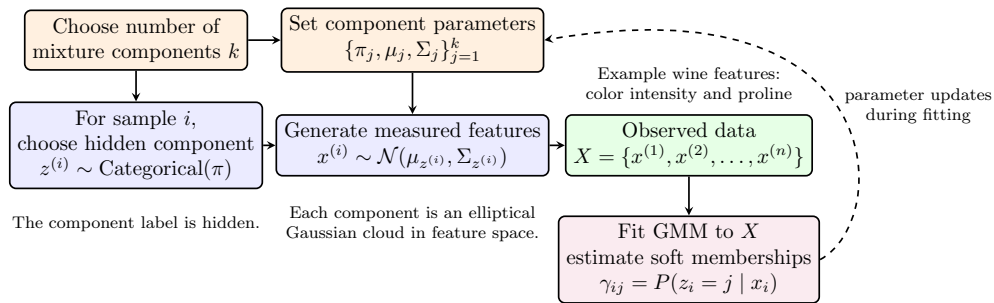
- ▶ π_j : mixing weight
- ▶ $\boldsymbol{\mu}_j$: component mean
- ▶ $\boldsymbol{\Sigma}_j$: component covariance matrix

The weights satisfy:

$$\pi_j \geq 0, \quad \sum_{j=1}^k \pi_j = 1$$

$\boldsymbol{\Sigma}_j$ controls each component's spread, shape, and orientation.

GMM Generative and Fitting Flow



The model estimates hidden component memberships and component parameters.

Soft Cluster Assignments

A GMM assigns probabilities, not just hard labels.

The responsibility of component j for observation i is:

$$\gamma_{ij} = P(z_i = j \mid \mathbf{x}_i) = \frac{\pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}{\sum_{\ell=1}^k \pi_{\ell} \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_{\ell}, \boldsymbol{\Sigma}_{\ell})}$$

Responsibilities satisfy:

$$0 \leq \gamma_{ij} \leq 1, \quad \sum_{j=1}^k \gamma_{ij} = 1$$

A hard label can still be produced:

$$\hat{z}_i = \arg \max_j \gamma_{ij}$$

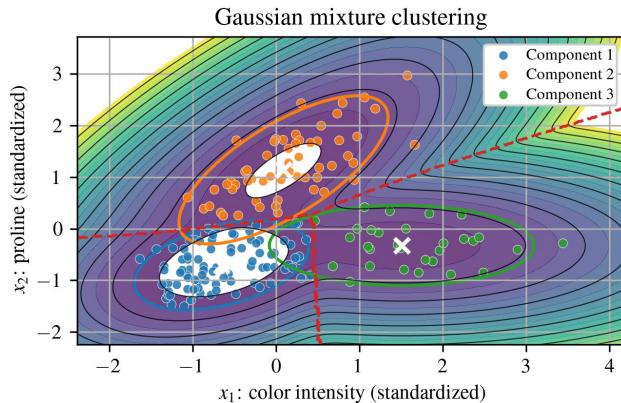
But the probabilities tell us how uncertain that assignment is.

Gaussian Mixture Clustering

The fitted GMM can be used to:

- ▶ assign each wine to its most likely component
- ▶ estimate component probabilities
- ▶ evaluate density or likelihood

Soft assignments are especially useful when components overlap.



GMM clustering using color intensity and proline.

Expectation-Maximization

GMM parameters are usually estimated using EM.

The objective is to maximize the log-likelihood:

$$\ell(\Theta) = \sum_{i=1}^n \log \left[\sum_{j=1}^k \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \right]$$

EM alternates:

E-step

- compute responsibilities γ_{ij}

M-step

- update weights, means, and covariances using weighted averages

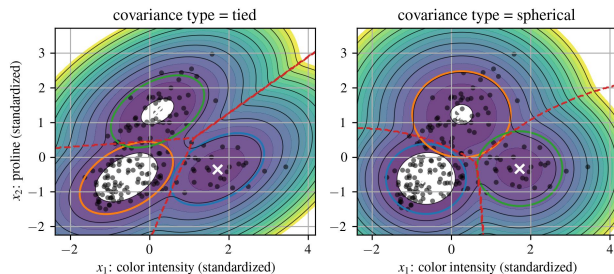
Repeat until the log-likelihood changes very little.

Covariance Structure

The covariance matrix controls the shape and orientation of each component.

Common choices:

- ▶ spherical: circular contours
- ▶ diagonal: different variance by feature
- ▶ tied: shared covariance
- ▶ full: separate tilted ellipses



Different covariance assumptions on the wine data.

Choosing the Number of GMM Components

GMMs also require choosing k , the number of components.

Because GMMs are probabilistic, likelihood-based criteria are common.

They balance:

- ▶ model fit
- ▶ model complexity

Two common criteria:

$$\text{AIC} = 2p - 2\ell(\hat{\Theta})$$

$$\text{BIC} = p \log(n) - 2\ell(\hat{\Theta})$$

Lower values indicate a better tradeoff.

BIC penalizes complexity more strongly when n is large.

Relationship Between K-Means and GMMs

K-means can be viewed as a limiting case of a Gaussian mixture model.

If all components have:

- ▶ spherical covariance matrices
- ▶ equal variance
- ▶ very small variance

then assigning a point to the most likely component becomes similar to assigning it to the nearest mean.

So K-means is like a:

hard-assignment, equal-variance version of a GMM

GMMs are more flexible, but they estimate more parameters.

Anomaly Detection

Clustering methods can also help identify observations that do not fit any group well.

These unusual observations may be called:

- ▶ anomalies
- ▶ outliers
- ▶ novelties

In engineering, anomalies may correspond to:

- ▶ faulty sensors
- ▶ rare operating conditions
- ▶ damaged components
- ▶ unusual loading events
- ▶ data-entry errors

Likelihood-Based Anomaly Detection

A simple idea:

1. fit a model to normal data
2. evaluate likelihood for each observation
3. flag low-likelihood observations

For a Gaussian model:

$$p(\mathbf{x}) = \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$$

Apply a threshold τ :

$$\hat{y}(\mathbf{x}) = \begin{cases} \text{anomaly,} & p(\mathbf{x}) < \tau, \\ \text{normal,} & p(\mathbf{x}) \geq \tau. \end{cases}$$

Equivalently:

$$\log p(\mathbf{x}) < \log \tau$$

Mahalanobis Distance

Gaussian anomaly detection is closely related to Mahalanobis distance:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

This measures distance from the mean after accounting for covariance.

Important idea:

- ▶ far along a high-variance direction may be normal
- ▶ moderate distance along a low-variance direction may be unusual

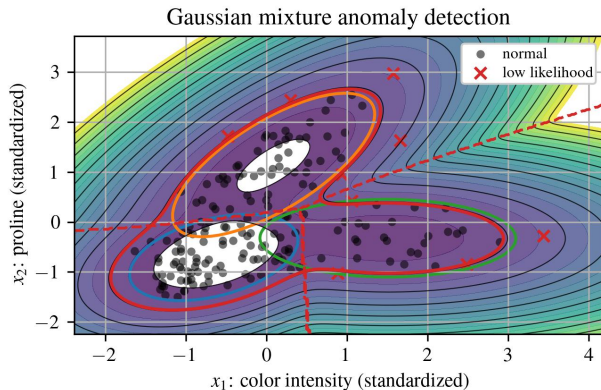
GMM Anomaly Detection

A GMM extends anomaly detection to data with multiple normal regimes.

A point is normal if it has high likelihood under at least one component.

A point is anomalous if it has low likelihood under the entire mixture.

The threshold controls the tradeoff between false alarms and missed anomalies.



Low-likelihood wine observations marked as potential anomalies.

Chapter 8 Takeaways

Different clustering methods answer the same question using different assumptions.

Method	Cluster idea	Watch out for
K-means	centroid distance	scale, k , spherical clusters
Hierarchical	nested merges	linkage and pairwise cost
DBSCAN/HDBSCAN	dense regions	density scale and metric
GMM	probability components	covariance, k , initialization
Anomaly detection	low likelihood	threshold choice

The final engineering step is always interpretation: connect the discovered structure back to the system that produced the data.